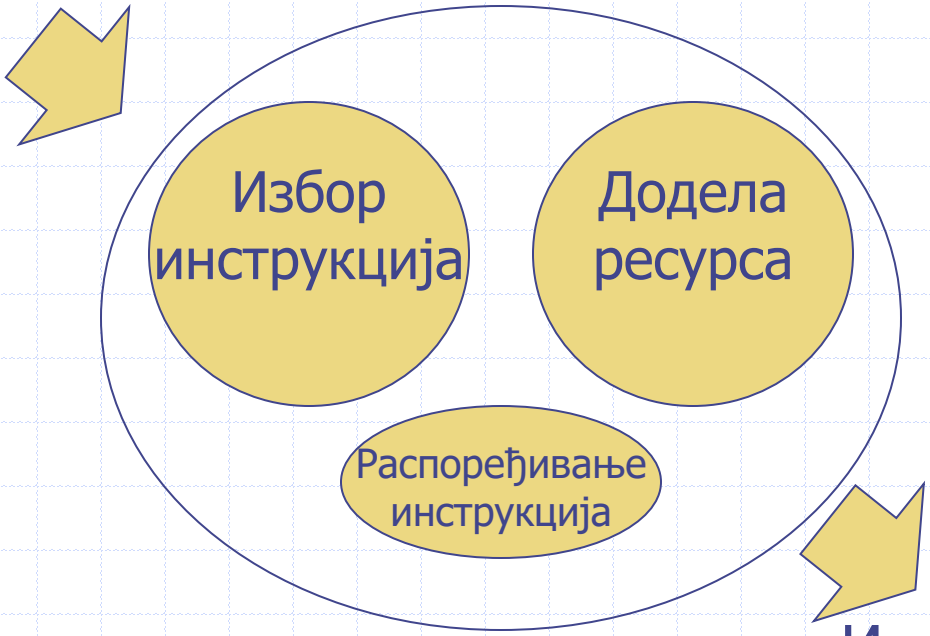
Поједностављени поглед на задњи део компајлера

Међурепрезентација
(Међујезик – IR)



Инструкције циљне архитектуре

Поједностављени поглед на задњи део компајлера

Међурепрезентација
(Међујезик – IR)



Инструкције циљне архитектуре

Поједностављени поглед на задњи део компајлера

Псеудо-асемблер
(листа инструкција које још увек не баратају стварним ресурсима)



Потпуни асемблерски код

Додела ресурса

- Потребно је операнде полазног језика (тј. међујезика) превести у операнде одредишног језика.

Операнди

- Операнди међујезика:
  - Променљиве, литерали...
- Операнди машинског језика:
  - Регистри
  - Меморија

Операнди

- Својства операнада међујезика:
- Својства регистара:
- Својства меморије:

Операнди

- Својства операнада међујезика:
  - Врста (ресурсни оп., константа/литерал...)
  - Тип (величина...)
  - Трајност
  - Назив
- Својства регистара:
  - Величина
  - Назив (редни број)
- Својства меморије:
  - Величина
  - Ширина

Може бити више група регистара!

Може бити више врста меморије! Логички (стек, општа меморија) и физички (различите меморијске банке)

Додела ресурса

- За сваки операнд међујезика потребна нам је одлука са којом врстом операнада одредишног језика да га заменимо.
- Та одлука зависи не само од својстава самог операнда, већ и од начина његове употребе и расположивог скупа инструкција.

Где сместити променљиву?

- Два питања:
  - Коју врсту ресурса доделити одређеној променљивој? (Тј. у коју врсту ресурса је сместити)
  - Који конкретан ресурс (или ресурсе) јој доделити? (Тј. у које је конкретне ресурсе сместити)

У коју врсту ресурса?

- Зависи од својстава променљиве и начина њене употребе.
- Примери:
  - Променљива статичке трајности -> меморија
  - Променљива аутоматске трајности ~> регистар
    - Параметар функције -> зависи од позивне конвенције
    - Са оваквом употребом: `foo(&x);` -> меморија
    - Великог типа -> меморија
- И још треба разматрати различите врсте регистара и меморије.

- У општем случају важи следеће:
  - Боље у регистар него у меморију
  - Сваки операнд може бити смештен у меморију
  - Меморије има довољно много

Додела меморије

- Једноставан приступ:

```
int x;      -> 4 бајта

MojTip y;   -> 8 бајтова

char z[5]; -> 5 бајтова

int a;      -> 4 бајта
```

Нпр. на MIPS архитектури

```
0000 x
0001 x
0002 x
0003 x
0004 y
0005 y
0006 y
0007 y
0008 y
0009 y
000A y
000B y
000C z
000D z
000E z
000F z
0010 z
0011 a
0012 a
0013 a
0014 a
```

Додела регистара

- Једноставан приступ:

```
int x;      -> 1 регистар

short y;    -> 1 регистар

int* z;     -> 1 регистар

int a;      -> 1 регистар
```

```
R0 Резервисан
R1 x
R2 y
R3 z
R4 a
```

Нпр. на MIPS архитектури

Додела регистара

- Једноставан приступ:

```
int x;      -> 1 регистар
short y;    -> 1 регистар
int* z;     -> 1 регистар
int a;      -> 1 регистар
int b;      -> 1 регистар
short c;    -> 1 регистар
int* d;     -> 1 регистар
int e;      -> 1 регистар
```

```
R0 Резервисан
R1 x
R2 y
R3 z
R4 a
R5 b
R6 c
R7 d
?? e
```

Има само 8 регистара!

Нпр. на MIPS архитектури

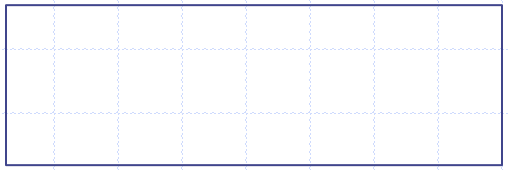
Анализа животног века програмских објеката

- Фаза избора инструкција не узима у обзир ограничен број регистара реалног процесора.
- Наравно, променљиве које нису истовремено у употреби могу делити исти регистар процесора.
- На тај начин је могуће велики број привремених променљивих смештати у расположиви скуп регистара процесора.

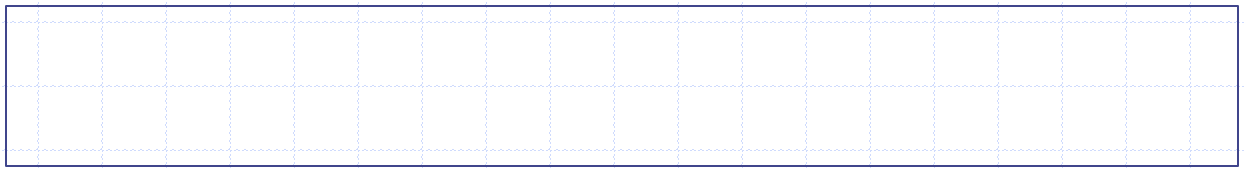
Преливање променљивих

- И поред дељења ресурса, број потребних регистара може превазићи број расположивих регистара. Решење је смештање прекобројних променљивих у следећу класу ресурса.
- За прекобројне променљиве се каже да су се **прелиле** у наредну класу ресурса.

Преливање - упрошћено

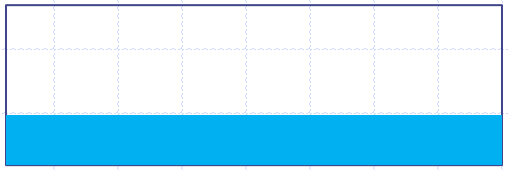


Регистри

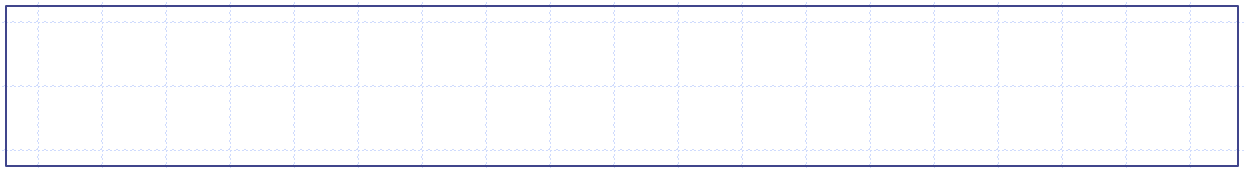


Меморија

Преливање - упрошћено

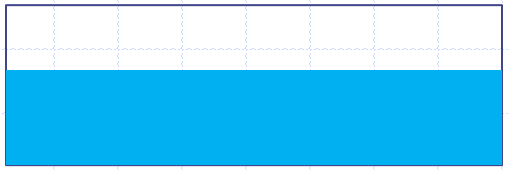


Регистри

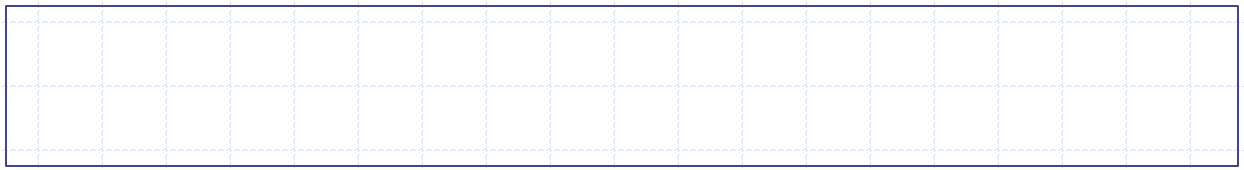


Меморија

Преливање - упрошћено



Регистри

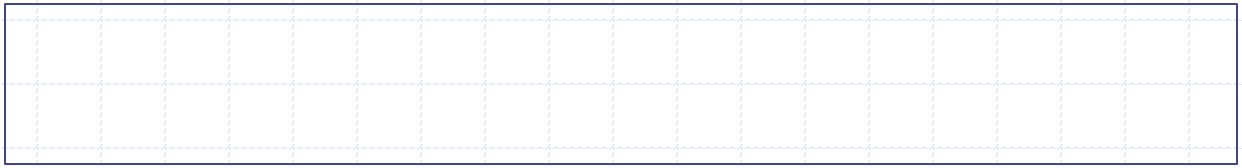


Меморија

Преливање - упрошћено

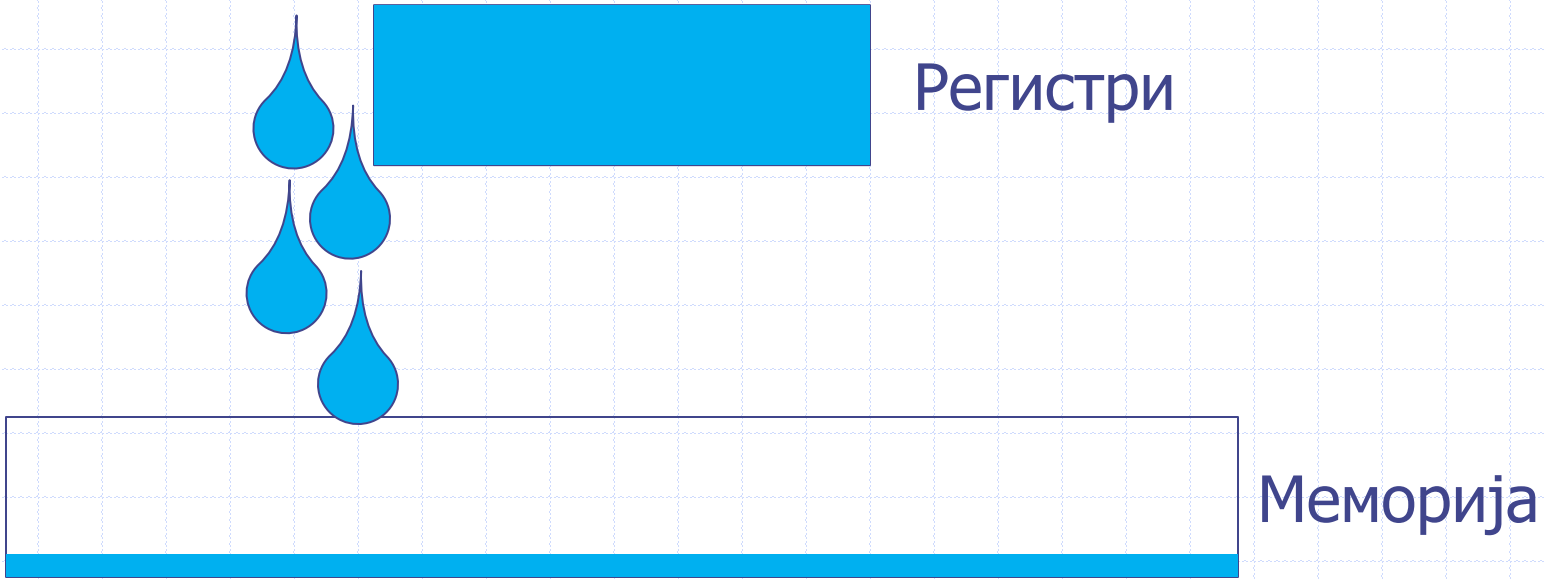


Регистри

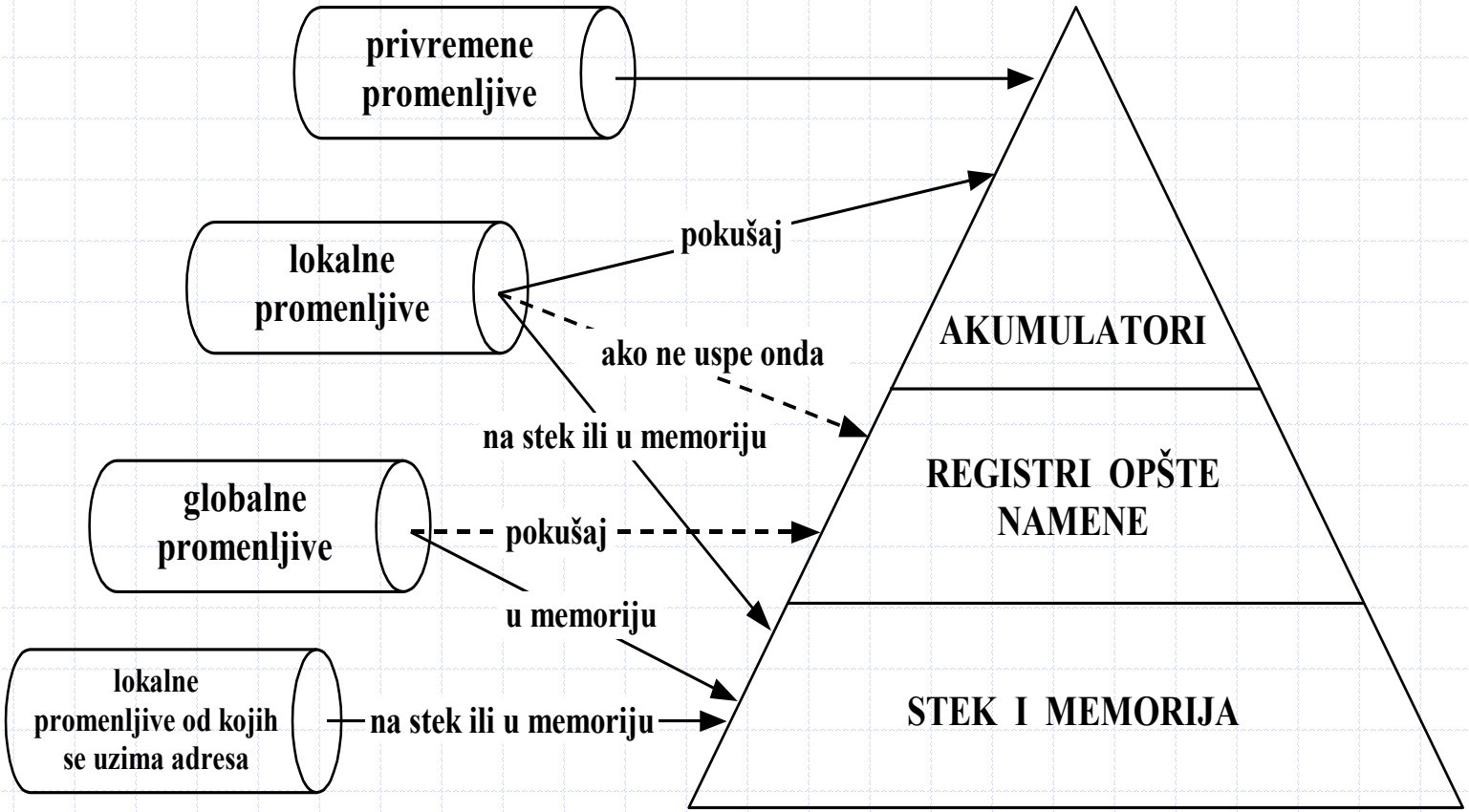


Меморија

Преливање - упрошћено



Која врста ресурса? Одговор за процесор MASC



Анализа животног века промењљиве

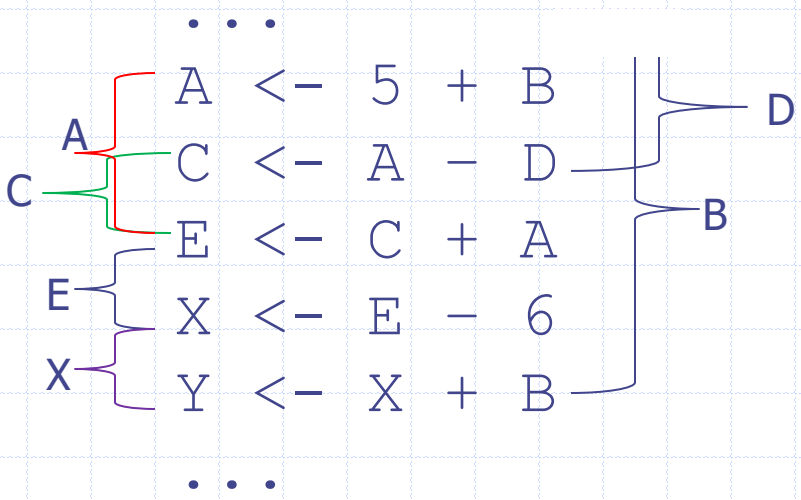
- Животни век промењљиве започиње првом дефиницијом промењљиве (упис почетне вредности у промењљиву), а завршава се њеном последњом употребом (последње очитавање садржаја промењљиве).

Животни век ових променљивих?

```
...
A <- 5 + B
C <- A - D
E <- C + A
X <- E - 6
Y <- X + B
...
```

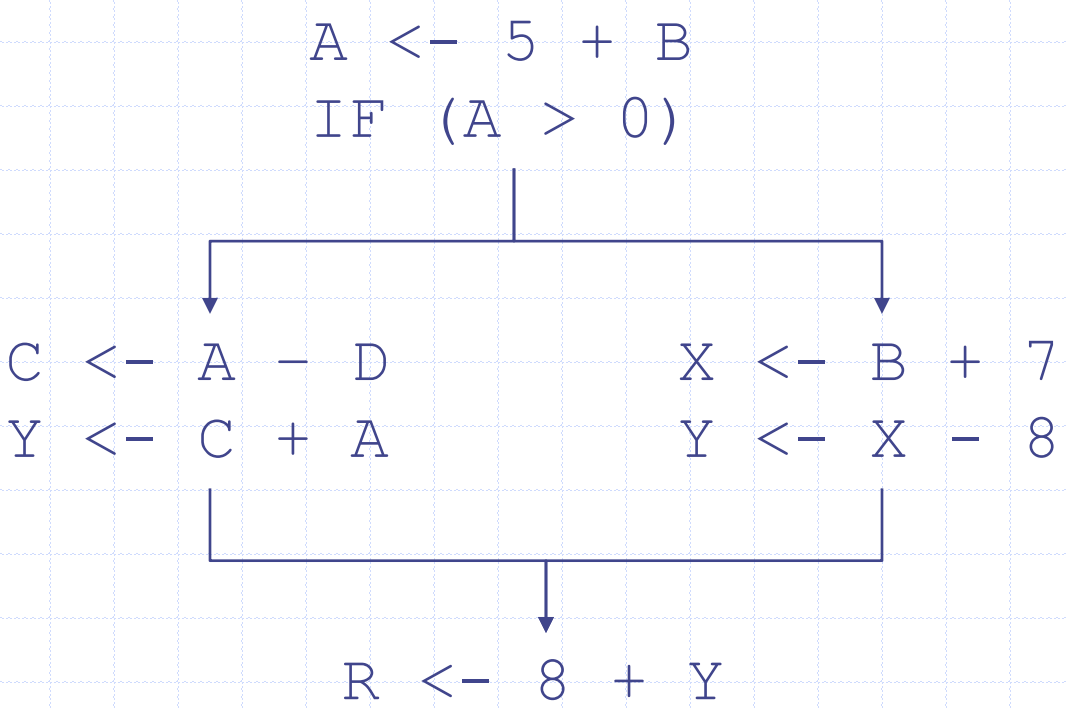
Које променљиве могу делити исти ресурс, а које не?

Животни век ових променљивих?



Које променљиве могу делити исти ресурс, а које не?

Животни век тече линијама извршавања



Зато је први корак у овој анализи конструисање графа тока управљања.

Граф тока управљања

- Свака инструкција програма представи се чвором графа, а затим се за сваке две инструкције x и y, такве да x претходи y, повлачи стрелица од x ка y.
- Често коришћен облик графа тока управљања је тзв. блок дијаграм алгоритма.

Пример графа тока управљања

$a \leftarrow 0$

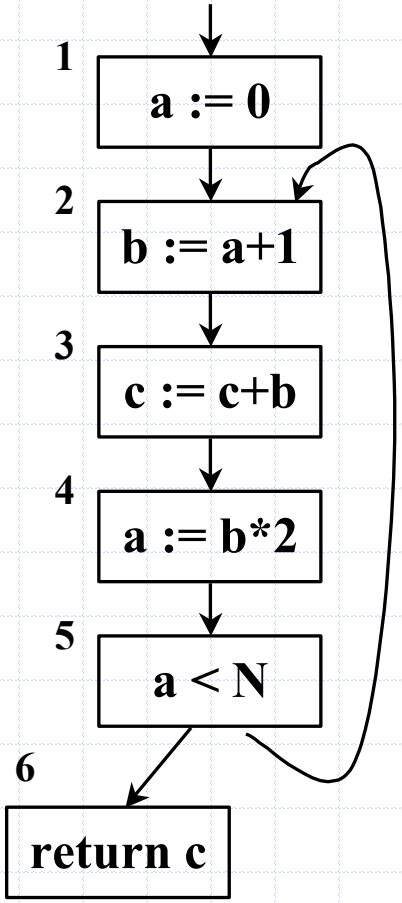
$L_1 :\ b \leftarrow a + 1$

$c \leftarrow c + b$

$a \leftarrow b * 2$

if $a < N$ goto L_1

return c



c је стално жива

b је жива на:

2 -> 3 -> 4

a је жива на:

1 -> 2 и

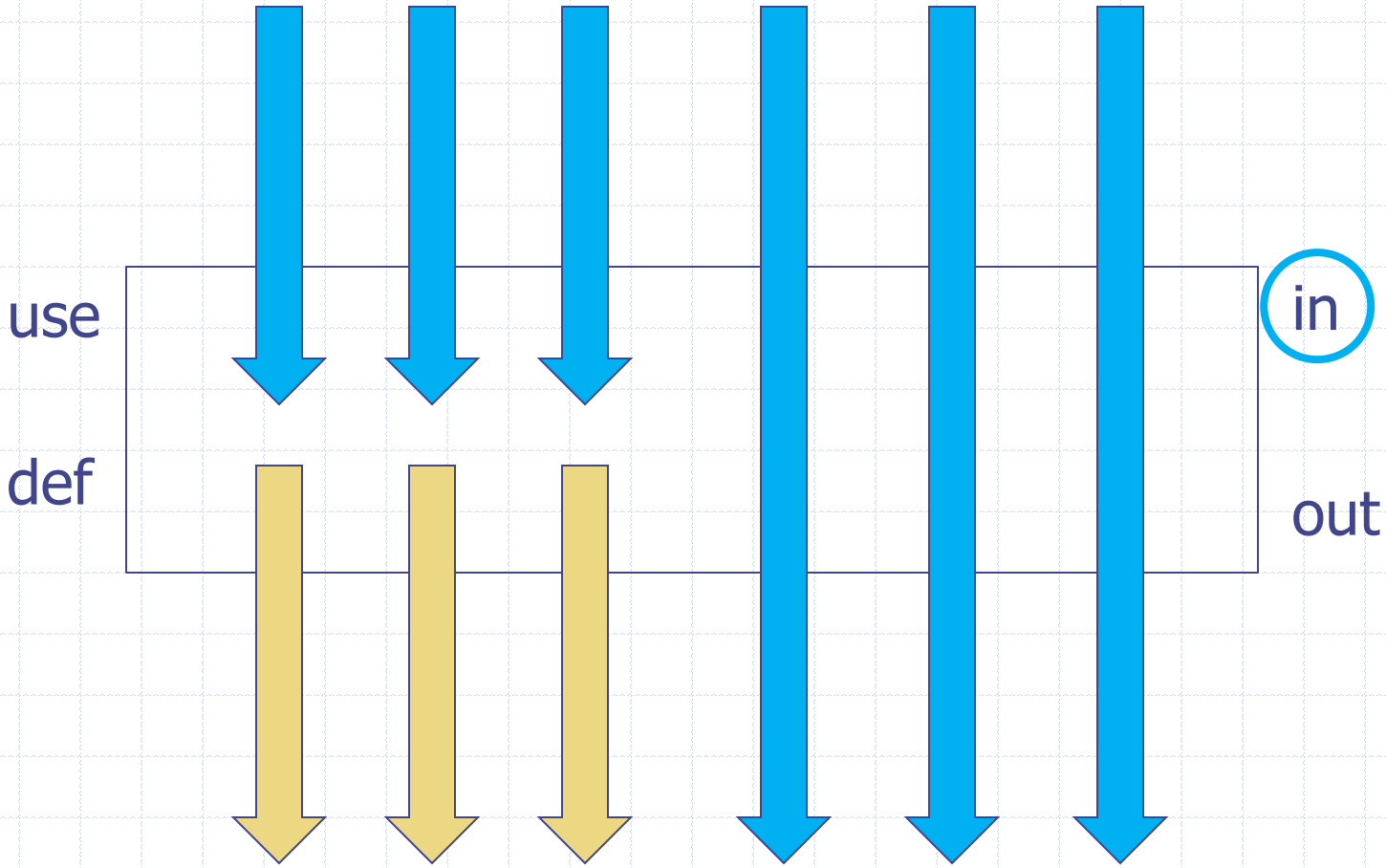
4 -> 5 -> 2

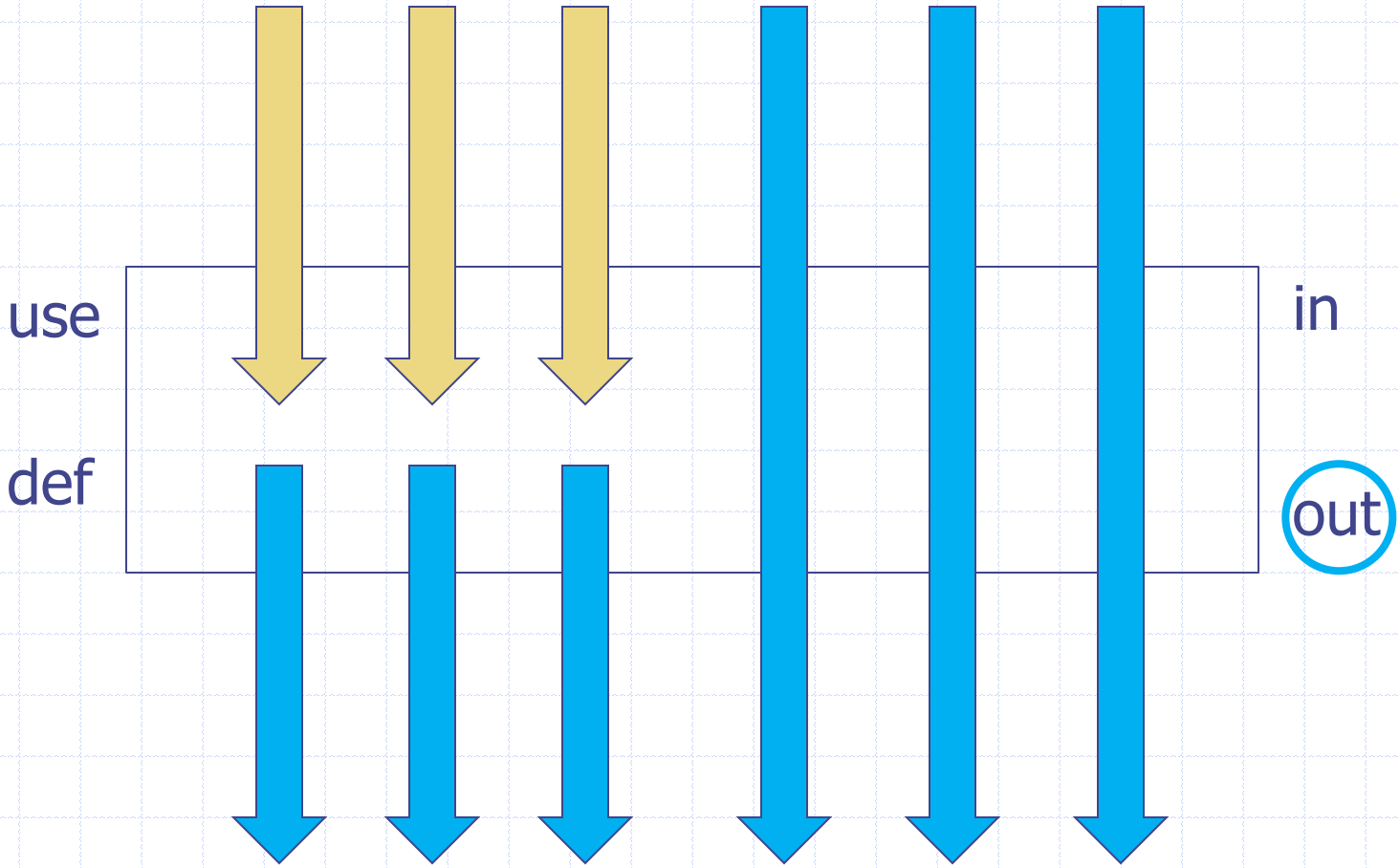
Граф тока података

- Чвор графа тока података има излазне стрелице које воде до чворова наследника и улазне стрелице које воде од чворова претходника.
- Скуп pred[n] је скуп свих чворова претходника за чвор n, а скуп succ[n] је скуп чворова наследника чвора n.
- Надаље се дефинише def[n] као скуп променљивих (операнада) које чвор дефинише (задаје им вредност), а use[n] као скуп променљивих које чвор користи (очитава).

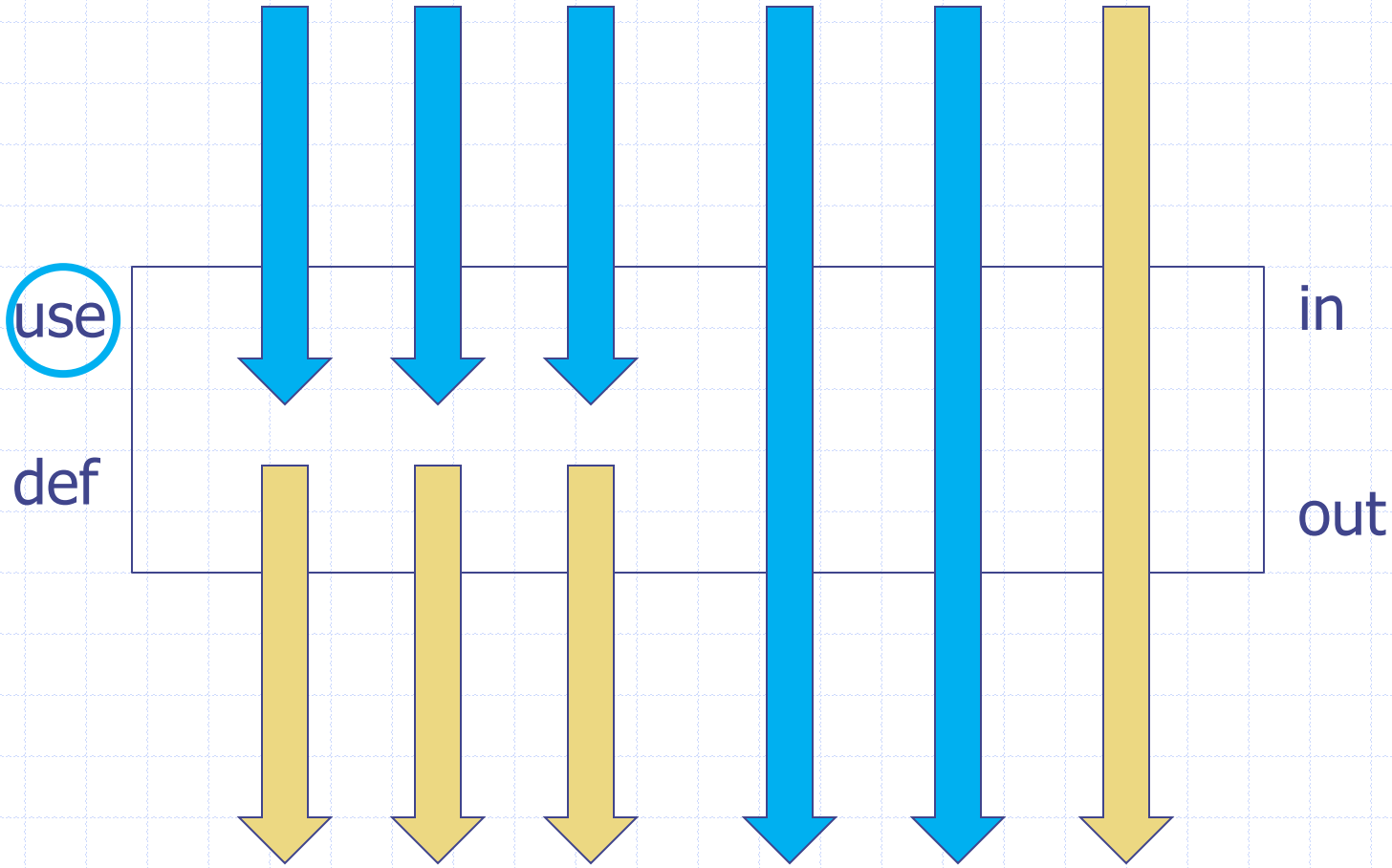
Жива на стрелици, Жива-на-улазу, Жива-на-излазу

- Променљива је жива на стрелици графа ако постоји усмерена путања од те стрелице до „use” чвора, а да путања не прелази преко иједног „def” чвора.
- Променљива је жива-на-улазу (енгл. live-in) чвора ако је жива на било којој улазној стрелици чвора.
- Променљива је жива-на-излазу (енгл. live-out) чвора ако је жива на било којој излазној стрелици чвора.

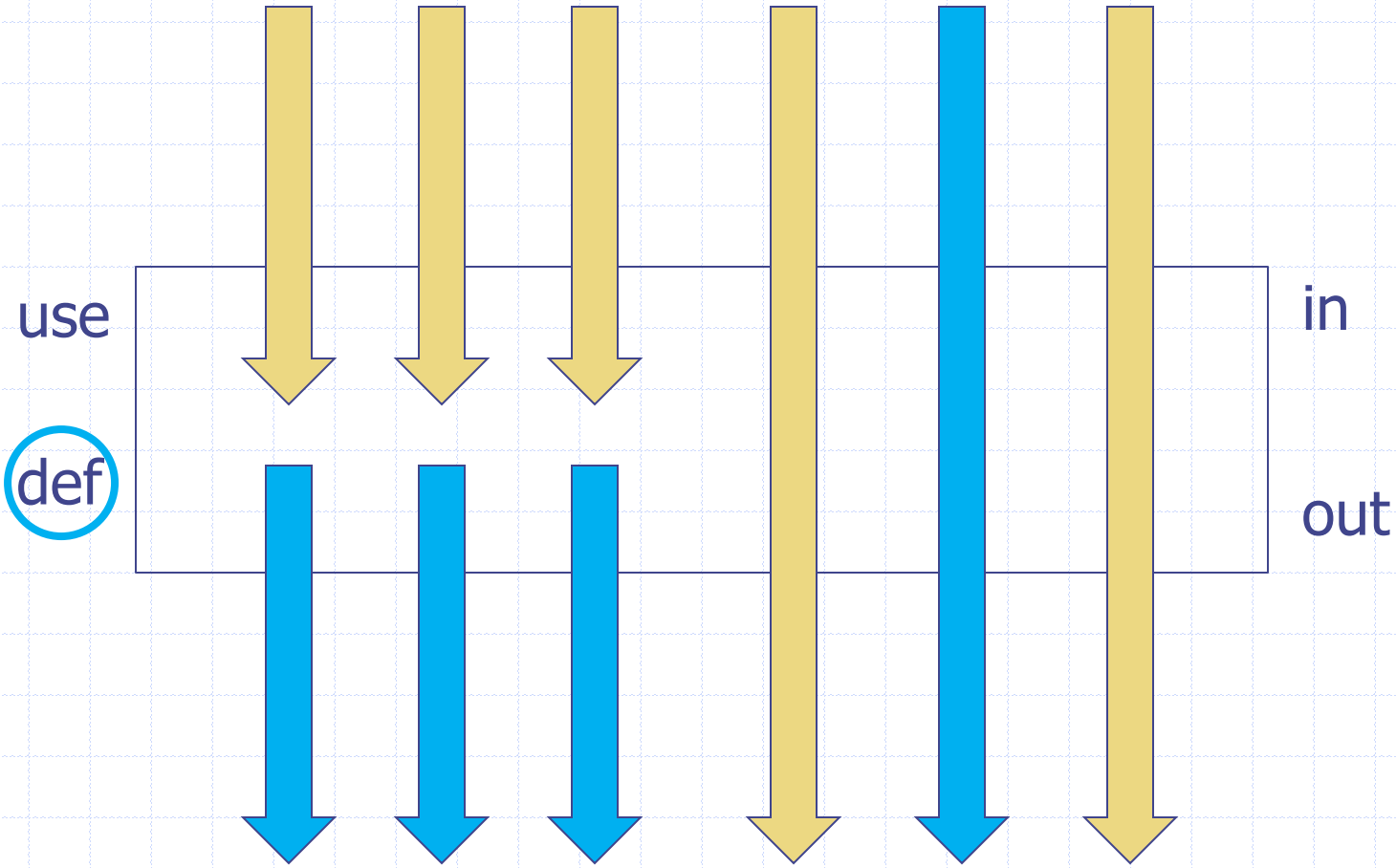

use
def
in
out


use
def
in
out

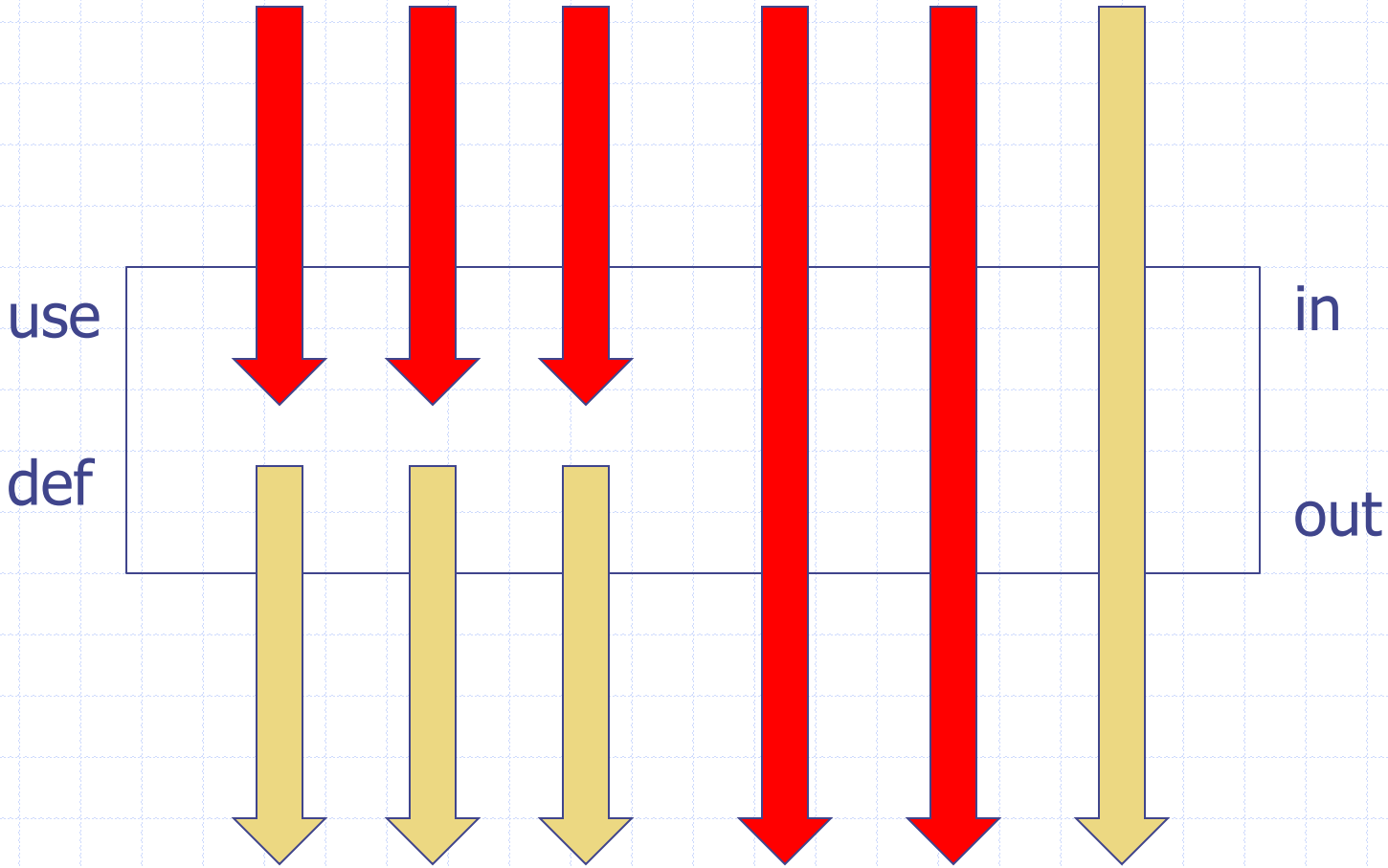
Јер не морају све променљиве чији животни век протиче кроз овај чвор да се користе у њему.



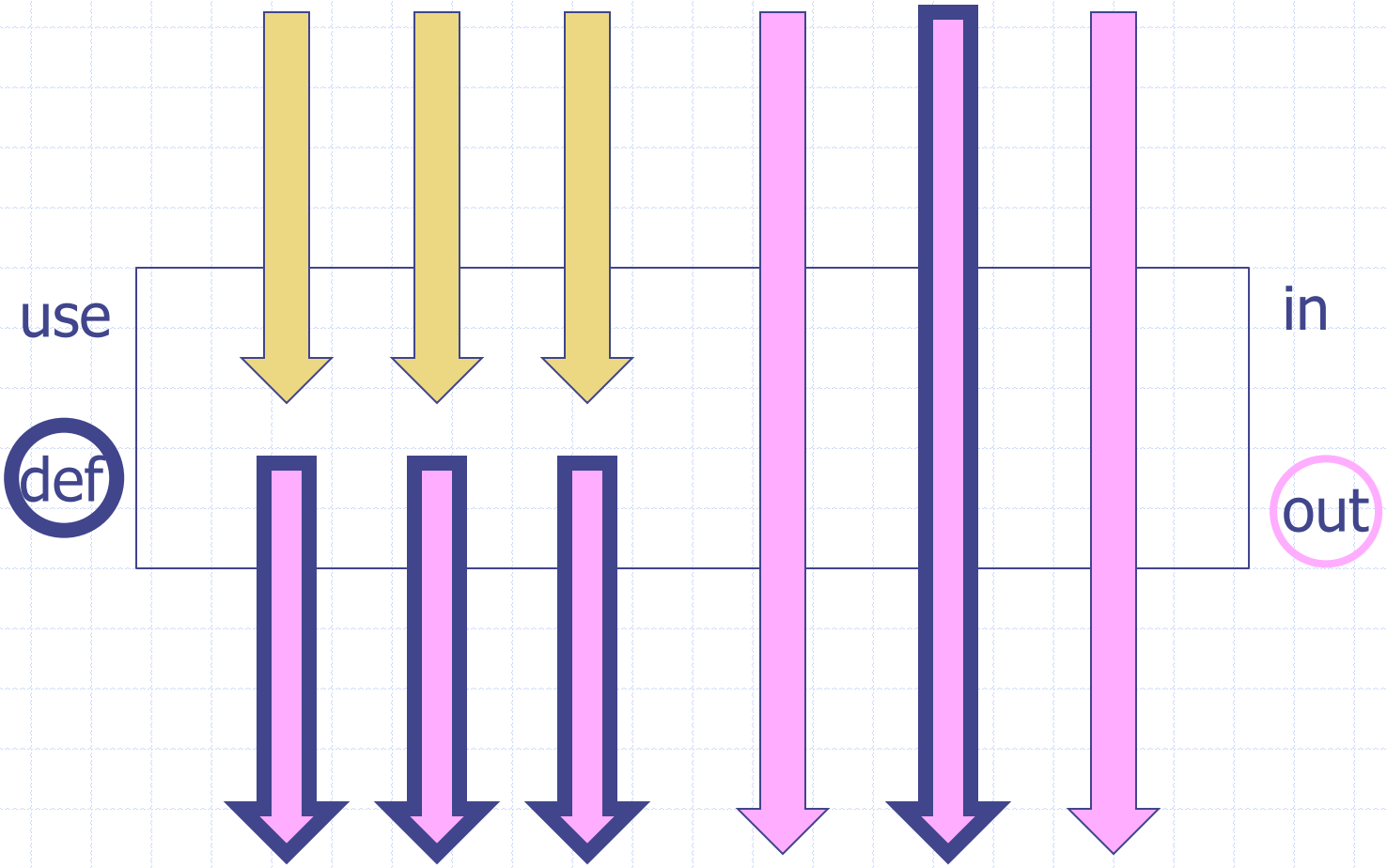
Неке промењљиве могу и да се користе и да се дефинишу и истом чвору.



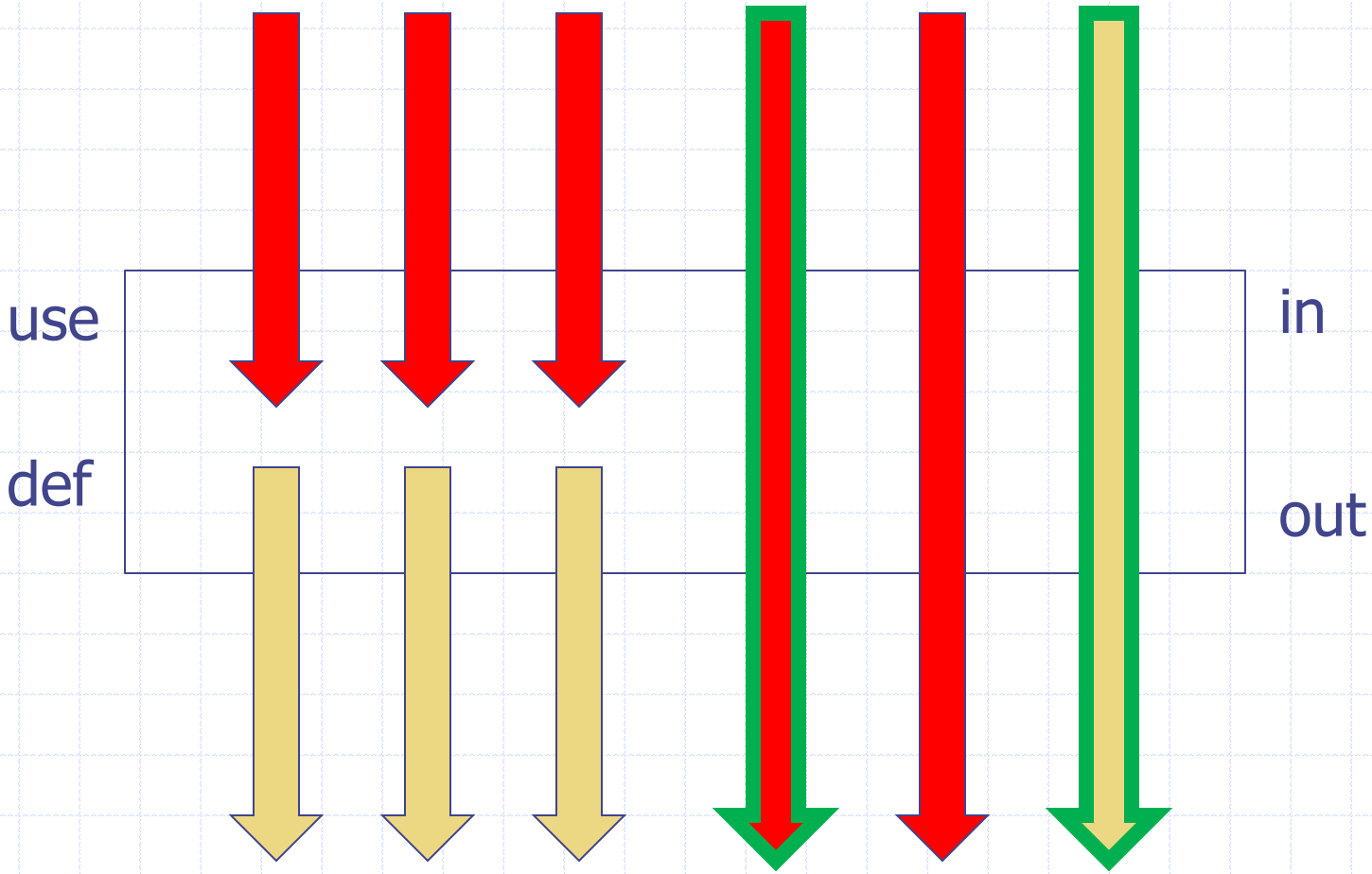
in = use ...



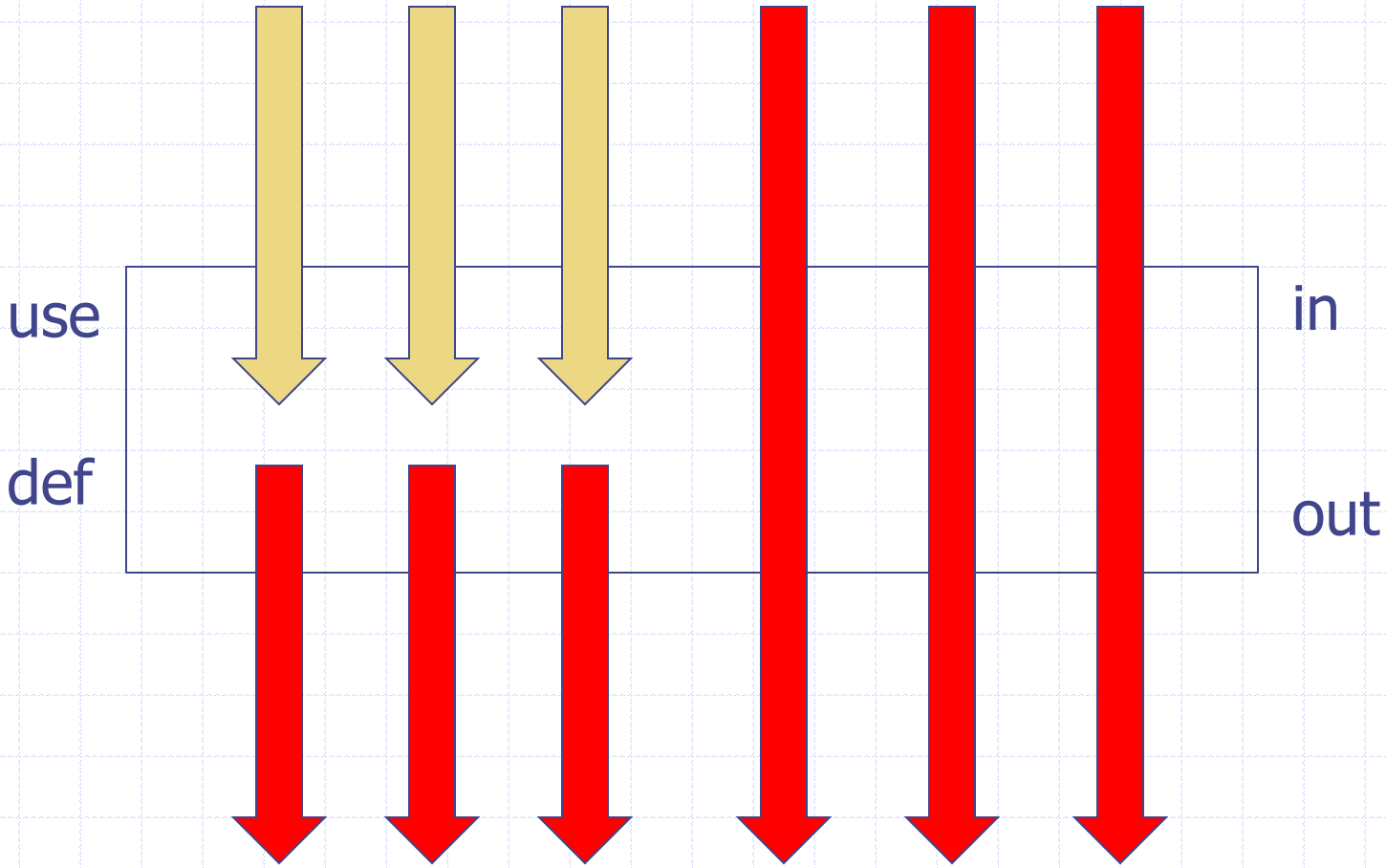
in = use ...



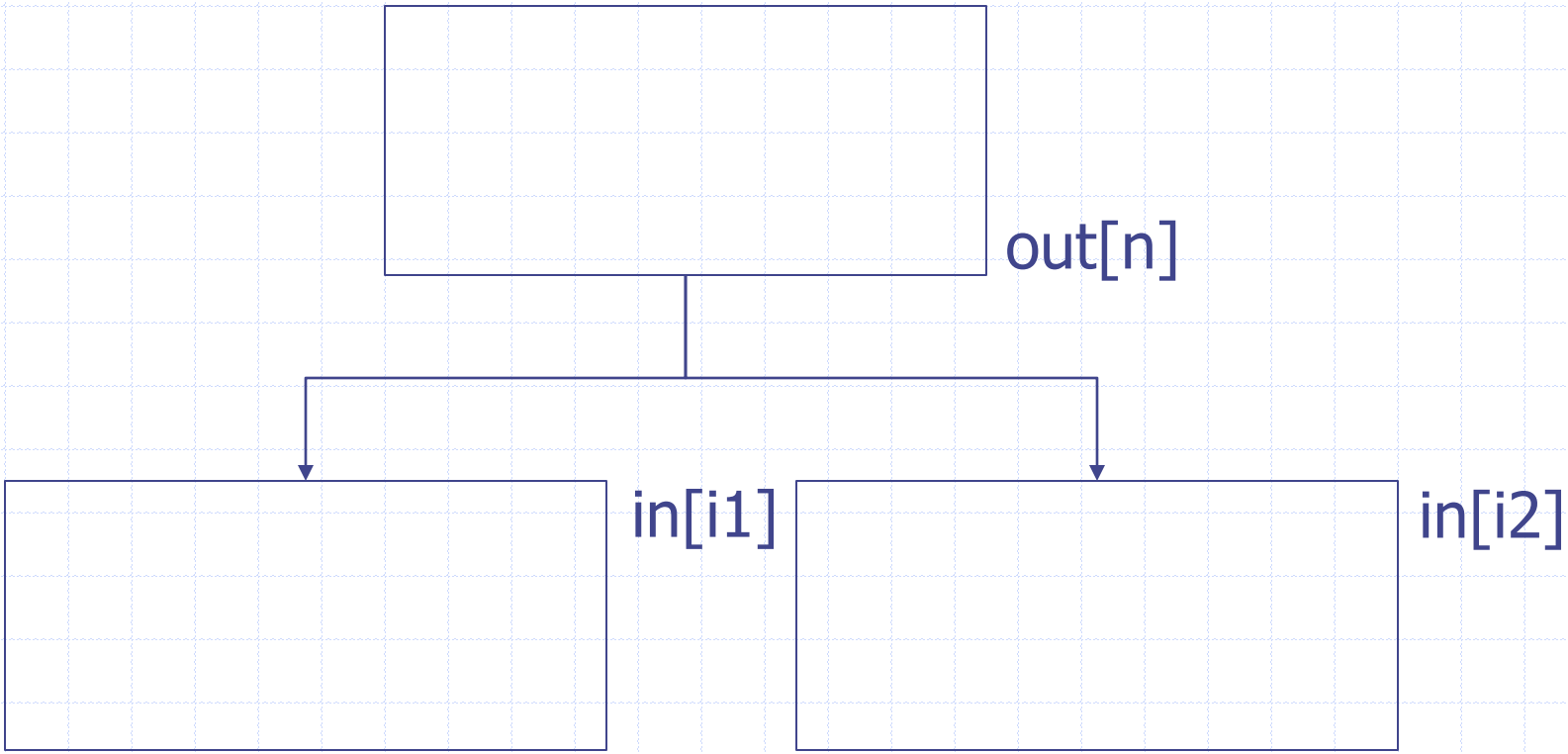
$$in = use \cup (out - def)$$



in = use U (out – def)
out = ...



in = use U (out – def)
out = U_{succ} in[s]



Једначине животног века

- Означимо скуп променљивих типа жива-на-улазу чвора n са in[n], а скуп променљивих типа жива-на-излазу чвора n са out[n]. Онда важи:

$$in[n] = use[n] \cup (out[n] - def[n])$$

$$out[n] = \bigcup_{succ[n]} in[s]$$

Једначине животног века

- За сваки чвор графа имамо по две једначине. Такође, за сваки чвор графа имамо две непознате. Ако је N број чворова, то је онда систем од 2N једначине са 2N непознатих.

Решење система једначина животног века

- Дато је следећим алгоритмом:

for each n
    in[n] = {}; out[n] = {}
repeat
    for each n
        in’[n] = in[n]; out’[n] = out[n]
        in[n] = use[n] U (out[n] – def[n])
        out[n] = Usucc[n] in[s]
until in’[n] = in[n] and out’[n] = out[n] for all n

Решење система једначина животног века

- Дато је следећим алгоритмом:

```
for each n
        in[n] = {}; out[n] = {}
repeat
        flag = false
        for each n
                in’ = in[n]; out’ = out[n]
                in[n] = use[n] U (out[n] – def[n])
                out[n] = Usucc[n] in[s]
                if in’ = in[n] and out’ = out[n]
                        flag = true
until flag = false
```

Практична реализација решења једначина животног века

- Прво се одређује out[n], а затим in[n].
- У овом примеру алгоритам конвергира брже за for(n=6;n>0;n--) него за for(n=1;n<7;n++). Прва петља конвергира у 3, а друга у 7 пролаза.
- Скупови се представљају низовима бита или сортираним листама променљивих.
- Унија скупова се одређује ИЛИ операцијом над низовима бита или спајањем листа уз одбацивање елемената који се понављају. Помаже STL.

Решење за дати пример

čvor	use	def	iteracija 1		iteracija 2		iteracija 3	
			out	in	out	in	out	in
6	c			c		c		c
5	a		c	ac	ac	ac	ac	ac
4	b	a	ac	bc	ac	bc	ac	bc
3	bc	c	bc	bc	bc	bc	bc	bc
2	a	b	bc	ac	bc	ac	bc	ac
1		a	ac	c	ac	c	ac	c

Сметња

- Једна од најважнијих примена резултата анализе животног века је обезбеђивање информације потребне за доделу ресурса (акумулатора, регистара опште намене итд.) променљивама.
- Проблем који онемогућава да се исти регистар додели двема променљивама назива се СМЕТЊА.
- Најчешћи узрок сметње је преклапање опсега животног века двеју променљивих.

Матрица сметњи

- Матрица сметњи има онолико врста и онолико колона колико има променљивих. У пресеку врста и колона за променљиве између којих постоје сметње уноси се ознака X.

Пример Матрице сметњи

	a	b	c
a			X
b			X
c	X	X	

Граф сметњи

- Граф сметњи је неусмерен граф, чији чворови одговарају променљивама, а два чвора графа се спајају неусмереним луком уколико постоји сметња између одговарајућих променљивих.

Правило за додавање нове сметње у граф

- За сваку дефиницију промењиве **a** додај сметње између **a** и сваке промењиве живе на излазу чвора.

Move операција као посебан случај

```
A <- 5 + B
C <- A
E <- C + A
```

```
A <- 5 + B
A <- A
E <- A + A
```

Правило би додало сметњу између A и C, али сметња је лажна.

```
A <- 5 + B
C <- A
C <- C + 1
D <- A - 3
E <- C * 8
```

Сметња не постоји због овога

Али постоји због овога

Правила за додавање нове сметње у граф

- За сваку деф. промењиве **a**, у чвору који није MOVE, додај сметње између **a** и сваке промењиве живе на излазу чвора.
- За MOVE инструкцију **a**=**c** додај сметње између променљиве **a** и сваке променљиве живе на излазу чвора која није **c**.